

Advantage Regression for Budget-Adaptive Depth Routing

HYUNGSEOP LEE¹, JIHO LEE¹, SUNWOO LEE¹, AND WOCHUL KANG¹ (Member, IEEE)

¹Department of Embedded Systems Engineering, Incheon National University, Incheon, Republic of Korea

Corresponding author: Wochul Kang (e-mail: wchkang@inu.ac.kr).

This paragraph of the first footnote will contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

ABSTRACT Dynamic-depth neural networks can trade accuracy for computation by selecting, per input, between a shallow (base) and a deep (super) execution path, but doing so requires a router that decides when the extra depth is worth its cost. Existing efficiency-aware routers are typically trained with a weighted sum of an accuracy term and a computation (FLOPs) term, frequently incorporating an additional uniformity or entropy term. This couples the operating point to training: the trade-off weight bakes a single accuracy–efficiency point into the model, while collapse onto one path must be prevented with auxiliary terms, and serving a new compute budget demands retraining. We reformulate depth routing as *advantage regression*. Instead of learning a discrete policy, a lightweight head predicts, for each image, how much detection loss the deep path would save over the shallow path. Consequently, routing reduces to simply comparing this predicted advantage against a threshold at deployment time. A single trained router therefore produces an entire continuous accuracy–efficiency Pareto curve through a simple threshold sweep, with no trade-off hyperparameter and no collapse-prevention term. On KITTI (trained on detection images, evaluated on tracking videos), the proposed router dominates random, luminance, edge-density, and confidence-based baselines, and the result transfers to BDD100K. The router adds negligible overhead, well under a thousandth of a percent of the detector computation, and yields measurable on-device latency, energy, and frame-rate improvements.

INDEX TERMS Adaptive inference, conditional computation, dynamic neural networks, efficient deep learning, object detection, accuracy–efficiency trade-off, edge AI

I. INTRODUCTION

Object detectors deployed on embedded and edge platforms operate under strict compute, latency, and energy constraints. Traditional detectors, however, are restricted to a single operating point, applying uniform computation to every frame regardless of scene difficulty or the optimal accuracy–efficiency trade-off. In practice, this trade-off varies substantially across inputs. For example, in autonomous driving, complex urban scenes containing dense traffic, occlusions, and numerous objects benefit from deeper processing to maximize detection accuracy. In contrast, highway scenes are often less cluttered and place a greater emphasis on low latency and high throughput, making shallower inference preferable. This input-dependent variation motivates the need for dynamic inference.

Recent advances, such as the AnyDepth-YOLO [1], enable this dynamic inference by embedding multiple execution depths within a single network. It exposes a *super* config-

uration (higher accuracy and computation) and a *base* configuration (lower computation and modest accuracy degradation). While this architecture provides the capacity for input-adaptive trade-offs, realizing these benefits at deployment requires a highly efficient *router* to determine, on a per-input basis, whether the additional computation of the super path is justified.

The central difficulty, however, lies in training this router. The dominant approach for learned routing trains a discrete policy using an efficiency-aware objective that linearly combines an accuracy term and a computation term, frequently incorporating an extra uniformity or entropy term. This formulation introduces three primary complications. First, the accuracy and efficiency terms are in direct conflict, and their balance is governed by a trade-off weight that *bakes a single operating point into the trained routing policy*; consequently, serving a new compute budget under this formulation requires

retraining the router. Second, because the routing decision is discrete and sampled (e.g., via Gumbel-Softmax or policy gradients), training is prone to collapsing onto a single path, which typically requires counteraction through auxiliary terms and their associated hyperparameters. Third, the sampled estimator injects high variance, which complicates optimization and can obscure which inputs truly warrant the deeper path.

We reframe depth routing as an *advantage regression* problem. Because the detector itself remains frozen, the benefit of selecting the deeper path for a given image is a fixed quantity: the per-image super advantage, defined as the reduction in detection loss achieved by the deep path relative to the shallow path. Instead of learning a discrete routing policy, a lightweight prediction head directly regresses this advantage. At inference time, routing reduces to comparing the predicted advantage against a threshold. This decouples training from the deployment operating point: a single trained router produces a continuous accuracy–efficiency Pareto curve through a simple threshold sweep, with no trade-off weight and no collapse-prevention term.

On KITTI, trained on detection images and evaluated on tracking videos, the proposed router Pareto-dominates random, luminance, edge-density, and confidence-based routing baselines, and the result transfers to BDD100K. The router adds negligible overhead, well under a thousandth of a percent of the backbone computation, while yielding measurable on-device latency, energy, and frame-rate gains; for instance, at a sweet-spot threshold it matches the deep path’s accuracy while reducing per-frame computation by about one sixth.

II. RELATED WORK

A. DYNAMIC AND ADAPTIVE INFERENCE

Adaptive inference reduces redundant computation by spending compute as a function of the input. Early-exit networks such as MSDNet [?], RANet [2], and EENet [3] attach intermediate classifiers and terminate once a confidence criterion is met. A complementary line equips a single backbone with multiple execution widths or depths, such as slimmable [?] and Once-for-All [4] networks, and layer- or block-skipping methods that drop residual units at inference [5], [6]. These approaches share a common goal: deciding *how much* compute to spend per input. Our work adopts the same dynamic-depth backbone (a shared base/super pair) but focuses on the orthogonal question of *how the routing decision itself is learned*.

B. LEARNED ROUTING WITH EFFICIENCY OBJECTIVES

A large body of work makes the routing decision *learnable* and trains it jointly with an efficiency penalty. Gumbel-Softmax-based routers such as AR-Net [7] sample discrete execution choices through a continuous relaxation [8]; gating and cosine-similarity controllers, NAS-style methods with cost-aware terms (e.g., DA-NAS [9]), and channel-selection approaches with activation/sparsity losses [?] follow the same template. These methods are almost universally trained with

a combined objective $\lambda_{\text{acc}}L_{\text{acc}} + \lambda_{\text{flops}}L_{\text{flops}} (+ \lambda_{\text{uni}}L_{\text{uni}})$, which has three documented limitations. (1) *Objective conflict*: the accuracy and efficiency terms pull in opposite directions, and their balance is fragile. (2) *Collapse*: a discrete, sampled policy tends to converge to a single action, requiring an entropy or uniformity term (and additional hyperparameters) to keep both paths alive. (3) *Single operating point*: the weight λ fixes one accuracy–efficiency trade-off in the trained weights, so serving a different compute budget requires retraining. Our regression formulation removes the efficiency weight and the anti-collapse term entirely: because the decision is recovered by thresholding a regressed scalar at deployment, a single model spans the whole trade-off family.

C. EFFICIENT OBJECT DETECTION

Efficient object detection has been pursued through lightweight backbones, quantization, and pruning, but these yield a static accuracy–efficiency point. We instead build on an AnyDepth-YOLO [1] whose backbone can be executed at two depths sharing the same weights, exposing a per-input depth choice at no extra storage cost. Detection is a particularly favorable setting for per-frame routing: in driving video the scene difficulty varies sharply over time (dense urban intersections versus empty highways, day versus night), so the value of the deep path is highly input-dependent [?], and a router can recover most of the deep path’s accuracy while running the shallow path on the majority of frames. This motivates evaluating routing not on still images but on tracking video, as we do on KITTI [10] and BDD100K [11].

III. METHOD

A. PROBLEM FORMULATION

AnyDepth-YOLO detector. We build on an AnyDepth-YOLO [1], a YOLOv12-style detector that is trained so that its depth can be reduced at inference while reusing the same weights. Each stage—in *both* the backbone and the detection neck—is split into an *essential* path that always runs and a *skippable* refinement path that can be dropped; skipping a stage’s refinement path therefore shortens the network end to end, not just in the backbone. This exposes, from a single set of parameters and with no extra storage, a family of execution depths. Following the detector’s design we use its two extreme configurations: a *super* path that executes every refinement path (highest accuracy, most compute) and a *base* path that skips all of them, keeping only the essential stages (shallowest, cheapest). Concretely, if the network has N skippable refinement paths across the backbone and neck, the super path uses the skip mask $[\text{false}]^N$ and the base path $[\text{true}]^N$; the detection head is shared and unchanged. The detector is trained once and then kept *frozen* in our work—we never modify its weights—so the only inference-time decision is which depth to run on each frame.

Routing problem. We thus view the detector as having two execution paths: a deep *super* configuration δ_{sup} and a shallow *base* configuration δ_{base} . For each input image, we define the

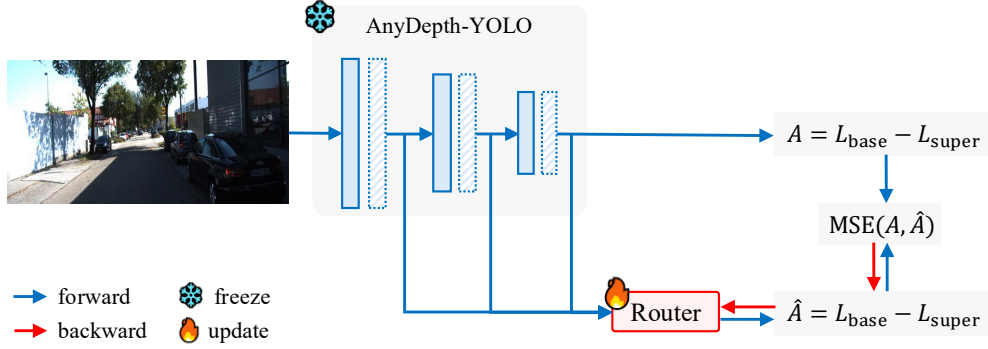


FIGURE 1. Training pipeline. The AnyDepth-YOLO detector is frozen; both paths are run once per image to obtain the true advantage $A = L_{\text{base}} - L_{\text{super}}$, and only the router is updated. The router predicts $\hat{A}$ from the multi-scale backbone features and is trained with $\text{MSE}(\hat{A}, A)$ (blue: forward, red: backward).

super advantage as the reduction in detection loss obtained by using the deep path:

$$A = L_{\text{base}} - L_{\text{super}}, \quad (1)$$

where L_{base} and L_{super} are the per-image detection losses of the two paths. At deployment, the predicted advantage $\hat{A}$ selects the execution configuration by a single threshold τ :

$$\text{config} = \begin{cases} \delta_{\text{super}}, & \hat{A} > \tau, \\ \delta_{\text{base}}, & \text{otherwise.} \end{cases} \quad (2)$$

B. ROUTER TRAINING

The router is trained entirely offline against the frozen detector (Fig. 1). We first cache, once, the features and the true advantage of every training image (Sec. III-B1), then fit a lightweight head (Sec. III-B2) to regress that advantage (Sec. III-B3); the detector is never updated.

1) Offline Feature and Loss Caching

The detector is kept *frozen* throughout: the router never changes the backbone, so for every training image the advantage in Eq. (1) is a fixed scalar that can be precomputed once. We exploit this by caching, offline, everything the router needs, which both decouples router training from the detector and makes each training epoch a cheap pass over precomputed tensors (no detector forward, so trade-off and hyperparameter sweeps are fast).

Concretely, for every image and for *each* of the two paths ($p \in \{\text{base}, \text{super}\}$) we run the frozen detector once and store: (i) multi-scale backbone features and (ii) the per-image detection loss. The features are tapped at the three backbone pyramid levels that feed the detection neck (the P3/P4/P5 stages, denoted $\{S_2, S_3, S_4\}$). For the GAP-MLP router we apply global average pooling to each level and concatenate, giving a single 768-d descriptor per image and path; for the TinyConv router we instead apply adaptive average pooling to a small $G \times G$ spatial grid (default 2×2) per level and concatenate along channels, giving a $768 \times G \times G$ tensor that preserves coarse spatial layout. We optionally also tap the neck (prediction) levels for the feature-source ablation. The

cached loss is the standard detector training loss (classification plus box and distribution-focal regression) evaluated per image against the ground-truth boxes; it correlates strongly with AP and serves as the supervision target through the advantage $A = L_{\text{base}} - L_{\text{super}}$. The result is a compact cache (train/val splits) of per-image, per-path feature tensors and losses on which the router is trained.

2) Router Architecture

The router takes the cached multi-scale features and outputs the scalar advantage prediction $\hat{A}$. How this head consumes the features is a design choice rather than a contribution of this work; we consider two variants and compare them as an ablation (Sec. IV-C4).

GAP-MLP. The 768-d global-average-pooled descriptor is normalized by a batch-normalization layer, projected by a linear layer with ReLU, and mapped to $\hat{A}$ by a small multilayer perceptron head. This is simple but discards all spatial layout. **TinyConv.** To retain “where in the frame the hard content is,” the router consumes the $G \times G$ feature grids instead of a pooled vector. Each feature group is processed by a tiny spatial projection: a 2D batch-normalization layer that equalizes the differing magnitudes of the multi-scale features before they are aggregated, a 1×1 convolution that reduces the concatenated channels to a small width with ReLU, a 3×3 depthwise convolution (stride 1, padding 1) with ReLU that mixes the coarse spatial cells, and finally global average pooling to a d -dimensional vector. The group vectors are concatenated and mapped to $\hat{A}$ by a two-layer head (hidden width 128). The depthwise convolution and the small channel width keep the router extremely cheap (Table 2) while the 2D normalization makes the multi-scale aggregation well-conditioned.

3) Training Objective

We train the router with a single regression objective on the cached advantages (Fig. 1):

$$L_{\text{acc}} = \text{MSE}(\hat{A}, A). \quad (3)$$

Unlike efficiency-aware routing objectives of the form $\lambda_{\text{acc}}L_{\text{acc}} + \lambda_{\text{flops}}L_{\text{flops}} (+ \lambda_{\text{uni}}L_{\text{uni}})$, our objective contains no

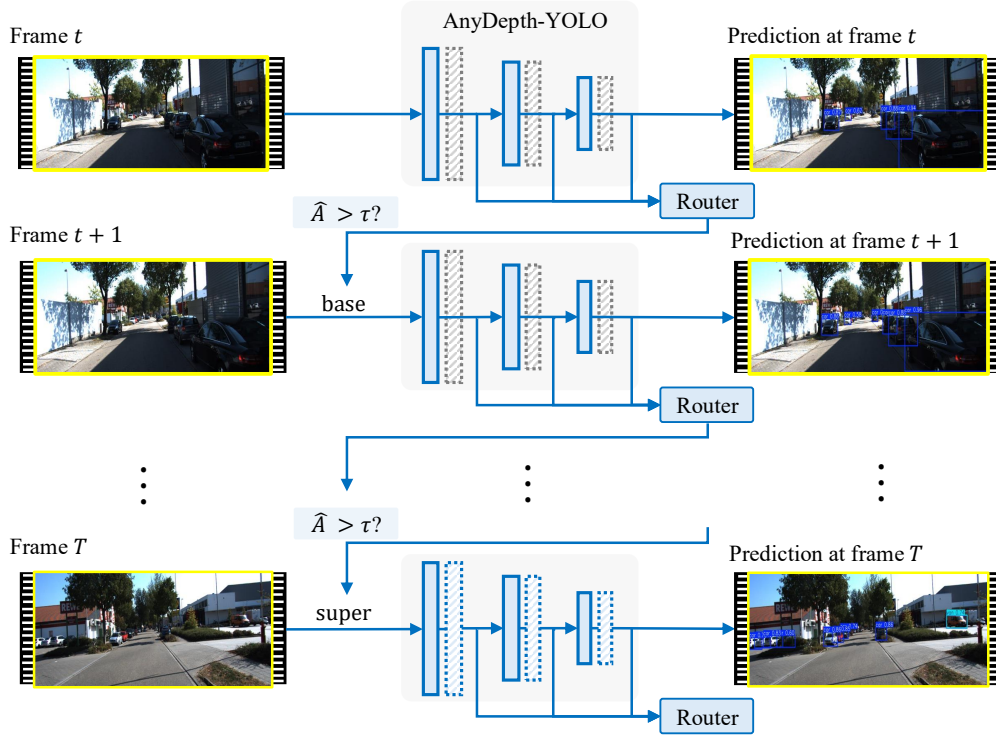


FIGURE 2. Deployment on video. At each frame the detector runs the chosen path and the router predicts $\hat{A}$ from its backbone features; the decision $\hat{A} > \tau$ selects the path (base or super) for the next frame. Routing is causal and recursive across frames $t, t+1, \dots, T$, adding only the lightweight router forward.

efficiency weight λ and no anti-collapse term, so the deployment operating point is not fixed during training. The advantage is optionally standardized (z-scored) across each mini-batch so that a few high-loss images do not dominate the gradient. Because at deployment the router reads the features of whichever path ran on the previous frame (Sec. III-C), training must expose it to both feature distributions: for each image we sample the assumed previous action as super with probability p (and base otherwise) and feed the corresponding cached features together with that action's identity. A balanced $p=0.5$ keeps the prediction calibrated regardless of which path executed upstream; we ablate p in Sec. IV-C3. Because the target A is a fixed property of each image under the frozen detector, the regression is a well-posed supervised problem rather than a non-stationary policy-learning one: there is no sampled action, hence no estimator variance and no risk of collapse, and no efficiency term that would tie the model to one budget. The router instead learns to *rank* images by how much the deep path helps. This ranking is what makes a single trained model span the whole trade-off family: sweeping the decision threshold over the range of predicted advantages moves the operating point continuously from all-base (largest threshold) to all-super (smallest threshold).

C. ROUTER DEPLOYMENT

Once trained, the router is deployed as a causal per-frame controller on top of the frozen detector. We first describe the basic threshold-based routing loop that serves a *fixed*

operating point (Sec. III-C1), and then an optional online extension that adapts the threshold to track a *time-varying* latency budget (Sec. III-C2).

1) Threshold-Based Routing

At deployment the routing rule of Eq. (2) is applied with a single scalar threshold τ . Because the router was trained only to predict the advantage, τ is a free parameter chosen at serving time: raising τ sends fewer images to the deep path (cheaper, lower accuracy) and lowering it sends more, so sweeping τ produces the continuous Pareto family from one model. A target compute budget is met simply by picking the corresponding τ ; in practice τ can be fixed on a held-out set so that no video labels are needed at serving time.

For video, routing is causal and per-frame (Fig. 2): the path for frame t is decided from features that are available before the heavy forward, namely the features produced by the path actually executed on frame $t-1$. Exploiting temporal continuity (adjacent frames have similar difficulty), this lets the router commit to a path at the start of each frame without any extra detector pass, so the only added cost is the lightweight router forward itself. Algorithm 1 states this loop: only the chosen path is run, and a fixed τ holds the system at one operating point on the Pareto front.

2) Online Latency-Budget Control

The fixed- τ loop serves one operating point. When the target latency budget itself varies over time—e.g. under thermal

Algorithm 1 Causal budget-adaptive depth routing (deployment)

Input: frozen AnyDepth detector D with base/super paths, router R , threshold τ , frames $x_1, \dots, x_T$
Output: streaming per-frame detections y_t

```

1  $prev \leftarrow \text{base}$   $\triangleright$  first frame defaults to base
2 for  $t = 1$  to  $T$  do
3    $(y, f) \leftarrow \text{run path } prev \text{ on } x_t \triangleright \text{detections + tapped features}$ 
4    $\hat{A} \leftarrow R(f, prev)$   $\triangleright$  predicted advantage  $L_{\text{base}} - L_{\text{super}}$ 
5    $a \leftarrow \text{super if } \hat{A} > \tau \text{ else base}$   $\triangleright$  Eq. (2)
6   if  $a \neq prev$  then  $(y, \cdot) \leftarrow \text{run path } a \text{ on } x_t$ 
7   emit  $y$  as the detection of frame  $t$ 
8    $prev \leftarrow a$ 
```

Algorithm 2 PI latency-budget control of the threshold τ

Input: target $L^*(t)$, path latencies $\ell_{\text{base}}, \ell_{\text{super}}, \ell_{\text{rtr}}$, gains k_p, k_i , EMA factor β

```

1  $\tau \leftarrow \tau_0; I \leftarrow 0; \bar{L} \leftarrow \frac{1}{2}(\ell_{\text{base}} + \ell_{\text{super}})$ 
2 for  $t = 1$  to  $T$  do
3    $a_t \leftarrow \text{route } x_t \text{ with threshold } \tau \triangleright \text{Alg. 1, lines 3–6}$ 
4    $\ell \leftarrow \ell_{a_t} + \ell_{\text{rtr}}$   $\triangleright$  realized per-frame latency
5    $\bar{L} \leftarrow \beta \bar{L} + (1 - \beta) \ell$   $\triangleright$  EMA of realized latency
6    $e \leftarrow L^*(t) - \bar{L}; I \leftarrow I + e$ 
7    $\tau \leftarrow \text{clip}(\tau - k_p e - k_i I) \triangleright \text{lower } \tau \Rightarrow \text{more super}$ 
```

or battery constraints—the same router supports tracking it online by adapting τ in closed loop, without any retraining. Algorithm 2 wraps a proportional–integral (PI) controller around the routing loop: it maintains an exponential moving average $\bar{L}$ of the realized per-frame latency, forms the error $e = L^*(t) - \bar{L}$ against the budget, and nudges τ by $-k_p e - k_i I$. Since lowering τ routes more frames to the deep path (raising latency) and raising it does the opposite, the controller drives the realized latency toward $L^*(t)$ at every frame. This optional layer reuses Algorithm 1 unchanged as its inner step; we evaluate its tracking behavior in Sec. IV-B4.

IV. EXPERIMENT**A. EXPERIMENTAL SETUP****1) Datasets**

We evaluate on two driving benchmarks, and on each the router is trained only on still detection images and tested on video, with no tracking or temporal supervision at any stage. *KITTI*. The router is trained on KITTI Detection [10] (80:20 split, 5985/1496 train/val images) and evaluated on the KITTI Multi-Object Tracking set (21 videos, 8008 frames), scoring the *car/pedestrian/cyclist* classes. Frames are processed at 384×1248 .

BDD100K. As a cross-dataset test, the router is trained on the BDD100K [11] detection images (70k train / 10k val, 10 classes) and evaluated on the BDD100K MOT validation videos (200 sequences labeled at 5 fps); routing and scoring run at the labeled 5 fps cadence, and we score the 8 movable classes the MOT split annotates (pedestrian, rider, car, bus, truck, bicycle, motorcycle, train). The MOT labels provide no tracks for the static *trafficlight/sign* classes that the 10-class detector

also predicts, so we exclude those two classes at scoring time to avoid penalizing every prediction as a false positive; this aligns the evaluated taxonomy with the labels and is applied identically to all strategies. Frames are processed at 720×1280 , and the underlying AnyDepth detector is the BDD-finetuned model, likewise kept frozen.

2) Video Evaluation Protocol

To compare every routing strategy on an identical detection pool, we run both the base and the super path on every frame and cache their detections; a strategy then differs only in which path’s detections it keeps for that frame and in how it makes that choice. This isolates the routing decision from detector variance, so all strategies (ours and the baselines) are scored in a single pass over the video and are directly comparable. Routing is causal and recursive: the decision for frame t is made from features produced by the path that was actually executed on frame $t-1$ (the first frame defaults to the base path), so no extra forward is needed to decide. Accuracy is reported as AP@.50 and AP@[.50:.95] obtained by matching the kept detections to the KITTI-tracking labels over all frames pooled. Per-frame computation is obtained from a precomputed lookup of the super/base FLOPs and aggregated by the realized SUPER-usage rate. On-device latency and energy are measured per frame on the target GPU with device synchronization and board-power sampling, after a warm-up phase; crucially, the router’s own forward is timed and charged to the routing strategy, so the reported latency, FPS, and energy include the router overhead.

This evaluation protocol mirrors the deployment routing loop (Algorithm 1), except that here both paths are cached every frame so all strategies share one detection pool; the latency-budget tracking experiment (Sec. IV-B4) likewise replays the PI controller of Algorithm 2.

3) Implementation Details

The router is trained for 300 epochs (batch 256, Adam, $1r \cdot 10^{-3}$) over seeds 0–4 and reported as mean $\pm$ std. The detector is frozen; only the router is updated, so each epoch is a cheap pass over the cached features. Detection uses confidence threshold 0.25 and NMS IoU 0.7.

4) Evaluation Metrics

We report AP@[.50:.95] and AP@.50, the SUPER-path usage rate, and GFLOPs/frame. On-device measurements (latency, FPS, energy) include the router forward and are reported against the super-only reference. Router cost is given as #params and FLOPs (Table 2).

B. RESULTS**1) Main Result: Pareto Dominance**

We compare our advantage-regression router against four routing strategies that share the same frozen detector and differ only in the per-frame signal used to choose a path: random routing, luminance- and edge-density-based routing,

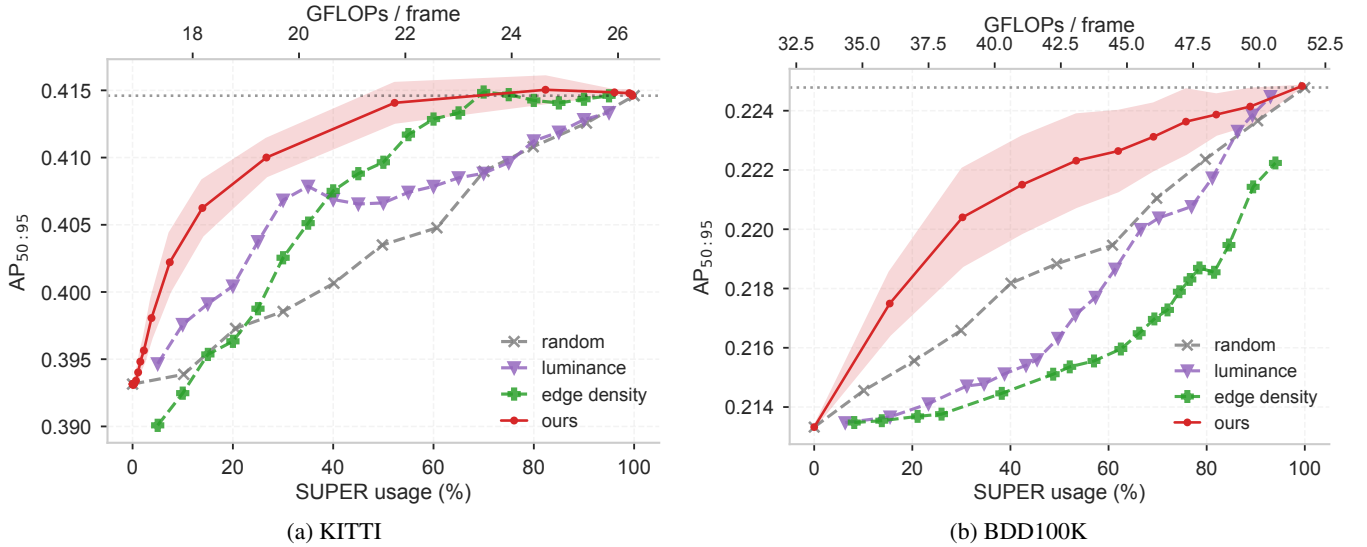


FIGURE 3. Depth routing (AP@[.50:.95] vs. SUPER usage): the proposed advantage-regression router (threshold sweep; TinyConv head, 2×2 grid) vs. random, luminance, and edge-density baselines, on (a) KITTI-tracking and (b) BDD100K MOT (cross-dataset, router trained only on still images). The single trained router traces a continuous Pareto front that dominates all baselines, most strongly in the low-budget (low SUPER-usage) region.

TABLE 1. Efficiency across every threshold τ in Fig. 3 (TinyConv router; latency, FPS, energy on an RTX 3090, including the router forward). AP is AP@[.50:.95] (%). a-base/a-super are the static single-path configurations (the endpoints of the curve in Fig. 3; $\tau \rightarrow \pm\infty$). Absolute latency/energy are not comparable across datasets (BDD100K uses a larger input and $\approx 2 \times$ the GFLOPs); only the within-dataset trade-off is meaningful.

(a) KITTI							(b) BDD100K						
τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)	τ	SUPER%	GFLOPs	AP	Latency (ms)	FPS	Energy (mJ)
always-base	0.0	16.87	39.3	13.35	74.9	2139	always-base	0.0	33.18	21.3	24.76	40.4	8389
+0.70	1.5	17.01	39.5	13.80	72.5	2213	+0.069	15.4	36.03	21.7	27.25	36.7	9145
+0.50	3.8	17.23	39.8	13.96	71.6	2239	+0.057	30.2	38.78	22.0	29.39	34.0	9872
+0.40	7.4	17.57	40.2	14.24	70.2	2283	+0.051	42.4	41.04	22.2	31.14	32.1	10468
+0.30	13.9	18.18	40.6	14.73	67.9	2362	+0.046	53.4	43.08	22.2	32.72	30.6	11008
+0.20	26.7	19.39	41.0	15.69	63.7	2515	+0.042	62.0	44.67	22.3	33.96	29.4	11427
+0.10	52.2	21.80	41.4	17.61	56.8	2823	+0.038	69.2	46.00	22.3	34.99	28.6	11779
+0.00	82.3	24.64	41.5	19.87	50.3	3185	+0.034	75.8	47.23	22.4	35.95	27.8	12104
-0.10	96.1	25.93	41.5	20.91	47.8	3351	+0.030	82.0	48.37	22.4	36.84	27.1	12407
-0.20	99.0	26.21	41.5	21.13	47.3	3386	+0.023	88.9	49.66	22.4	37.83	26.4	12746
-0.40	99.6	26.27	41.5	21.17	47.2	3394	-0.027	99.4	51.62	22.5	39.35	25.4	13263
always-super	100.0	26.30	41.5	20.87	47.9	3343	always-super	100.0	51.72	22.5	39.16	25.5	13289

and confidence routing (the mean of the top-20 box scores, or of the scores ≥ 0.1 , on the previous frame); each is swept over its decision threshold to trace a curve. Fig. 3 shows that on *both* datasets the single trained router traces a continuous Pareto front that dominates every baseline across the operating range, while the baselines reach super-only accuracy only near 100% SUPER usage (i.e., with no savings). On KITTI (Fig. 3(a)) the router holds AP@[.50:.95] within 0.1 point of the super-only ceiling (41.4 vs. 41.5) while routing only 52% of frames to the deep path, i.e. at 21.8 rather than 26.3 GFLOPs/frame (-17% compute); pushing to 27% SUPER usage costs just 0.2 AP for a 26% compute reduction, whereas at the same budgets the strongest baseline still trails the super-only line by 1–2 points. On BDD100K (Fig. 3(b)) the curve has the same shape: at only 30% SUPER usage the router already recovers the bulk of the deep-path gain (22.0 vs. 22.5 AP@[.50:.95]) at 25% less compute. On both, the margin over the baselines is widest in the low-budget (low SUPER-usage)

region, where choosing *which* frames deserve the deep path matters most. Notably, BDD100K is a cross-dataset test—the router is trained only on still BDD images with no tracking or temporal supervision—yet it transfers to the MOT video unchanged, confirming that the formulation is not KITTI-specific.

Table 1 confirms that these FLOP savings translate into real on-device gains once the router forward is charged to the budget. At the KITTI sweet-spot ($\tau = +0.10$) the 17% compute cut yields 16% lower latency and energy and 19% higher FPS at no accuracy loss; on BDD100K the 30%-SUPER operating point gives 25% lower latency, 26% lower energy, and 34% higher FPS. The router overhead (Table 2) is below 0.002% of the SUPER path, so the measured savings track the FLOP reduction almost exactly.

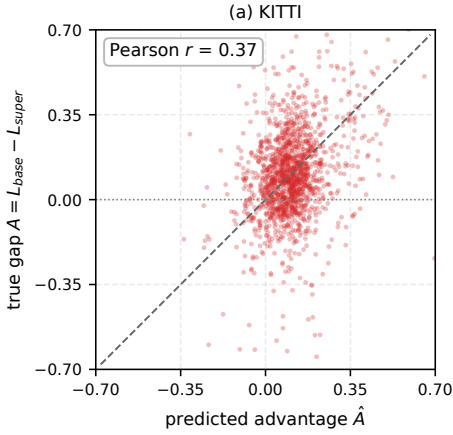


FIGURE 4. Predicted advantage $\hat{A}$ vs. true loss gap $A = L_{\text{base}} - L_{\text{super}}$ on the KITTI validation set (5-seed ensemble). The router orders images by how much the deep path helps.

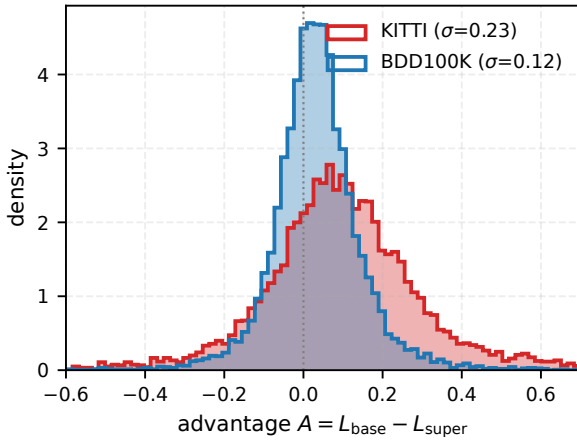


FIGURE 5. Per-image advantage $A = L_{\text{base}} - L_{\text{super}}$ distribution. BDD100K is concentrated near zero (small base $\rightarrow$ super gap, $\sigma=0.12$), KITTI is roughly twice as wide ($\sigma=0.23$); the smaller, noisier BDD100K gap is why feature source barely matters and the operating range is spanned by a narrow τ band there.

2) Advantage Regression Quality

Fig. 4 relates the router's predicted advantage $\hat{A}$ to the true per-image loss gap A on the validation set (Pearson $r=0.37$). The correlation is positive but deliberately modest, and this is expected rather than a weakness: the regression target is a *single-image* loss difference, an intrinsically noisy quantity (the same scene yields a different gap under small appearance changes), so no predictor—however expressive—can correlate tightly with it. What the router must get right is not the exact magnitude but *which* frames benefit most from the deep path, and the prediction is compressed toward the mean precisely because it averages out this label noise. The decisive evidence that the signal is good enough is downstream: thresholding this $\hat{A}$ yields the continuous, baseline-dominating Pareto front of Fig. 3 on both datasets—an operating-point quality that a poorly ordered score could not produce.

TABLE 2. Router overhead (per frame, batch 1). Both routers add negligible cost relative to the SUPER path (26.30 GFLOPs).

Router	#Params	GFLOPs	% of SUPER
GAP-MLP	60,241	1.17×10^{-4}	0.0004
TinyConv (2×2)	60,881	4.17×10^{-4}	0.0015
TinyConv (4×4)	60,881	1.61×10^{-3}	0.0060

3) On-Device Efficiency

4) Runtime Latency-Budget Tracking

The threshold τ exposes the operating point as a single scalar knob, so the detector can be *steered at runtime* to follow a time-varying latency budget $L^*(t)$ (e.g. a thermal- or battery-driven schedule) without any retraining. We close a simple control loop around τ : a PI controller compares the target to an exponential moving average of the realized per-frame latency and nudges τ to cancel the error (lowering τ admits more SUPER frames, raising latency; raising it does the opposite). Because per-frame cost is memoryless, we replay the cached per-frame advantages and the measured path latencies to evaluate the loop. Fig. 6 shows that the realized latency tracks both a step and a sawtooth budget tightly (mean abs. error 0.21 and 0.23 ms) while detection accuracy is held ($\text{AP@} [.50:.95] \approx 44.9$). A single trained router thus turns the continuous Pareto front into a runtime-adjustable budget the system can steer frame-by-frame.

C. ABLATION STUDIES

1) Input Feature Level: Backbone vs. Neck vs. Both

We compare three feature sources for the router on both datasets: the backbone pyramid features (input-level representations, before the detection neck), the neck features (prediction-level representations, immediately before the heads), and their concatenation. On KITTI backbone features win and adding the neck does not help (backbone $>$ both $>$ neck; Fig. 7), and Table 3 shows the same conclusion holds on BDD100K, so we adopt a single backbone-only router throughout.

The two feature sources answer different questions. Neck features encode *what the detector currently predicts* under the executed path; they are informative about confidence on *that* path, but the quantity we need is *counterfactual*—how much the *other*, deeper path would improve the result. That gap is largely determined by raw scene properties (clutter, object scale, lighting) that are most cleanly exposed in the input-level backbone features and are partially washed out by the time the signal reaches the prediction-level neck. Concatenating the neck therefore adds variance without adding discriminative signal, and dilutes the clean backbone cue. This is the same lesson that lets purely pixel-level signals (edge density, luminance) route well [?]: the benefit of going deep is written into the image content itself, so input-level features are the right place to read it—*pixels know when to go deep*.

Quantitatively (Table 3) the ordering is *dataset-dependent*

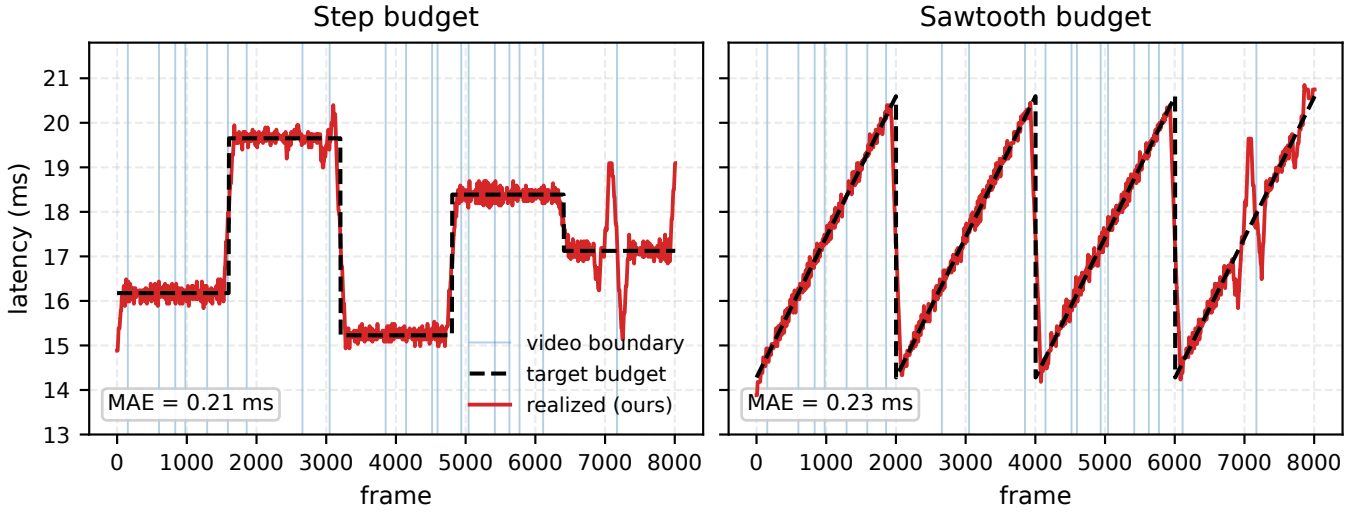


FIGURE 6. Runtime latency-budget tracking on KITTI-tracking. A PI controller adjusts the router threshold τ each frame so the realized per-frame latency (moving average, red) follows a time-varying target $L^*(t)$ (black), for a step (left) and a sawtooth (right) budget; tracking error is ≈ 0.2 ms. Latency uses the measured RTX-3090 path anchors (Table 1a).

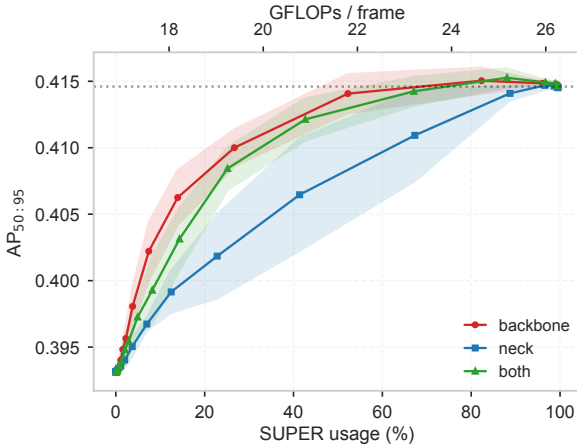


FIGURE 7. Feature ablation ($AP@.50:.95$): feature source (backbone/neck/both).

in magnitude but not in conclusion: on KITTI backbone features win clearly (a 0.42-point AUC margin over neck, $\approx 5 \times$ the seed std), whereas on BDD100K all three sources are statistically indistinguishable (< 1 std apart). The difference is explained by the routing headroom: the BASE $\rightarrow$ SUPER accuracy gap is only ≈ 1.2 points on BDD100K versus ≈ 2.2 on KITTI, and the per-image advantage is correspondingly smaller and noisier—its distribution is concentrated near zero on BDD100K ($\sigma=0.12$) and roughly twice as wide on KITTI ($\sigma=0.23$; Fig. 5), which is also why the useful τ band is narrow on BDD100K and broad on KITTI (Table 1). When the exploitable signal is this small, every reasonable feature source saturates the same low ceiling and the choice of tap can no longer differentiate; KITTI, with more exploitable structure, rewards the cleaner input-level cue. Either way backbone-only is *best-or-tied on both datasets* and is also the cheapest (no neck tap, smallest router), which is why we make

TABLE 3. Feature-source comparison across datasets: AUC of the $AP@.50:.95$ -vs-SUPER usage curve (percent, mean $\pm$ std over 5 seeds). Backbone is the best-or-tied choice on both, motivating a single backbone-only router.

Feature source	KITTI	BDD100K
backbone (input)	41.12 ± 0.07	22.12 ± 0.02
neck (pred)	40.70 ± 0.08	22.12 ± 0.01
both	41.03 ± 0.08	22.14 ± 0.01

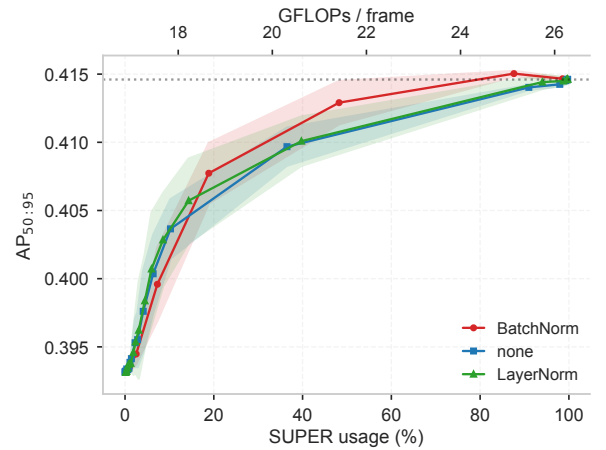


FIGURE 8. Feature normalization (none/BN/LN). $AP@.50:.95$.

it the single consistent design.

2) Feature Normalization: None vs. BatchNorm1d vs. LayerNorm

Fig. 8).

We compare no normalization, BatchNorm1d, and LayerNorm on the pooled multi-scale descriptor (Fig. 8). The three pyramid levels $\{S_2, S_3, S_4\}$ live at very different acti-

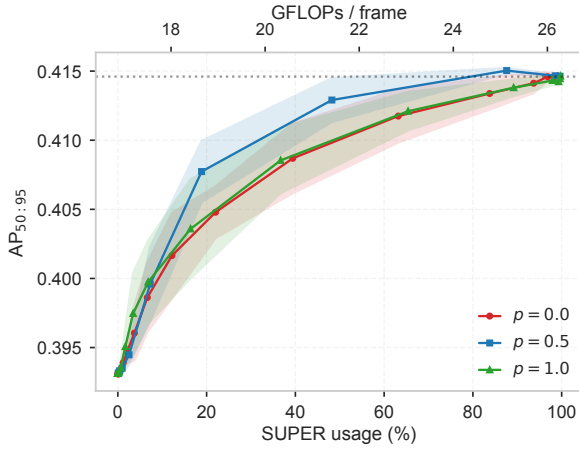


FIGURE 9. Previous-action sampling ratio $p(\text{super})$ during training (AP@.50:.95).

vation magnitudes, so without conditioning the largest-scale features dominate the regression and the smaller scales are effectively ignored. The decisive question is *along which axis* the statistics are removed. BatchNorm normalizes each feature *across images* in the batch, which equalizes the multi-scale magnitudes while *preserving the relative differences between images*—and it is precisely those between-image differences that encode which frames are hard. LayerNorm instead normalizes *within each image* across features, which removes exactly the global “how active is this whole image” signal that correlates with difficulty, and can therefore erase the cue the router needs. In short, normalize across images (keep the difficulty ranking), not within an image (which destroys it); BatchNorm is the safe default for this reason, with the additional benefit of well-conditioned multi-scale aggregation.

3) Previous-Action Sampling Ratio $p(\text{super})$

During training we sample the assumed previous action (the path whose features the router conditions on) as super with probability p , sweeping $p \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ (Fig. 9). We find $p=0.5$ best, with the symmetric extremes tied and worse ($p=0.5 > 0.25=0.75 > 0.0=1.0$).

This is a train/serve distribution-matching effect. At deployment the router must read features produced by *whichever* path ran on the previous frame—sometimes base, sometimes super—and the same image yields a slightly different feature map under each path. If training only ever shows super-path features ($p=1.0$), the router is never calibrated to base-path features and mis-ranks frames whenever the previous frame ran base; $p=0.0$ fails symmetrically. Training with a balanced $p=0.5$ exposes the router to both feature distributions equally, so its advantage prediction stays calibrated regardless of which path was executed upstream. The takeaway is that the previous action is part of the router’s input distribution, and balanced sampling is what makes the causal, per-frame deployment of Sec. III-C robust.

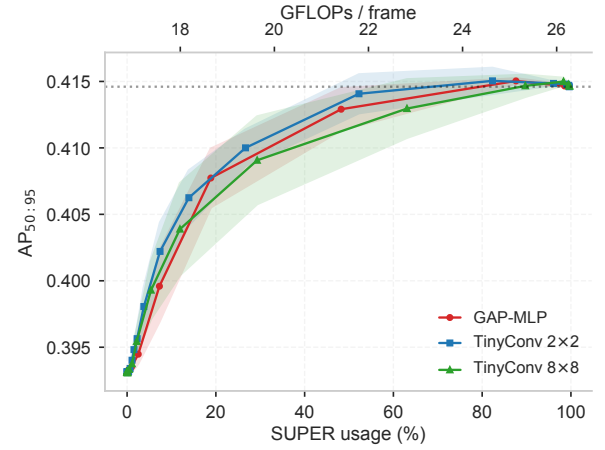


FIGURE 10. Router design and grid resolution. GAP-MLP is compared with TinyConv at 2×2 and 8×8 grids in AP@.50:.95.

4) Router Design and Grid Resolution

We compare GAP-MLP with TinyConv at 2×2 and 8×8 grids (Fig. 10). TinyConv, which preserves coarse spatial layout, Pareto-dominates GAP-MLP: at $\tau=+0.10$ it matches the same AP@.50:.95 using 52% SUPER versus GAP-MLP’s 65%. Grid resolution beyond 2×2 does not help, and the 2×2 TinyConv is the strongest setting. GAP collapses the whole frame to one vector, so it can only ask “is this image hard on average?” But difficulty is usually *localized*: a single dense cluster of small objects in one corner of an otherwise empty road is what makes the deep path pay off, and that spatial concentration survives in a 2×2 grid but is averaged away by global pooling. This is why even a tiny amount of retained spatial layout closes a large part of the gap to the deep path. At the same time, the cue is *coarse*—it is about *where roughly* the hard content sits, not its fine shape—so a 2×2 grid already captures it and a finer 8×8 grid only adds parameters and variance without new signal. The sweet spot is the smallest grid that preserves quadrant-level locality.

V. CONCLUSION

We reformulated dynamic-depth routing as advantage regression, decoupling the training objective from the deployment operating point. A single lightweight router predicts the per-image super advantage from cached backbone features, and a threshold sweep recovers an entire continuous accuracy–efficiency Pareto curve with no trade-off weight and no anti-collapse term. On KITTI-tracking the router Pareto-dominates random, luminance, edge-density, and confidence baselines and matches super-only accuracy at a 17% FLOPs (and 16% energy) reduction, at negligible overhead. A limitation is that the regression view targets the two-path (super/base) decision; extending it to multi-level or per-layer (2^N) routing, where the decision space is no longer enumerable and sampling- or marginalization-based estimators become necessary, is left to future work.

REFERENCES

- [1] W. Kang, H. Lee, and J. Lee, "Anydepth-detr/-yolo: Any-depth object detection with a single network," *arXiv preprint arXiv:2605.09407*, 2026.
- [2] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, "Resolution adaptive networks for efficient inference," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2369–2378.
- [3] F. Ilhan, K.-H. Chow, S. Hu, T. Huang, S. Tekin, W. Wei, Y. Wu, M. Lee, R. Kompella, H. Latapie *et al.*, "Adaptive deep neural network inference optimization with eenet," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 1373–1382.
- [4] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once for all: Train one network and specialize it for efficient deployment," in *International Conference on Learning Representations*, 2020.
- [5] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "Skipnet: Learning dynamic routing in convolutional networks," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 409–424.
- [6] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "Blockdrop: Dynamic inference paths in residual networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8817–8826.
- [7] Y. Meng, C.-C. Lin, R. Panda, P. Sattigeri, L. Karlinsky, A. Oliva, K. Saenko, and R. Feris, "Ar-net: Adaptive frame resolution for efficient action recognition," in *European conference on computer vision*. Springer, 2020, pp. 86–104.
- [8] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=rkE3y85ee>
- [9] X. Dai, D. Chen, M. Liu, Y. Chen, and L. Yuan, "Da-nas: Data adapted pruning for efficient neural architecture search," in *European conference on computer vision*. Springer, 2020, pp. 584–600.
- [10] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [11] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "Bdd100k: A diverse driving dataset for heterogeneous multi-task learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.

• • •